

PyOmniGraph - Getting and hosting your own federated copies of subgraphs of Wikidata and other knowledge graphs^{*}

Wolfgang Fahl^{1,*}, Tim Holzheim¹, Christoph Lange² and Stefan Decker¹

¹*RWTH Aachen University, Computer Science i5, Aachen, Germany*

²*Fraunhofer FIT, Sankt Augustin, Germany*

Abstract

Wikidata is an example for a very large, crowd sourced general knowledge graph backed by a world wide community since 2012. Factgrid is using the same Wikibase infrastructure as Wikidata since 2017 for historical research projects. GOV (Geschichtliches/Genealogisches Ortsverzeichnis) is an online gazetteer for searching ancient places. Wikidata grew so large that the Blazegraph endpoint powering it was pushed to the 4 Terrabyte limit and the Wikimedia Foundation therefore decided to split the knowledge graph to host two instances. The challenges of federated queries that came up in the above use cases led to the need of being able to work with subgraphs of the RDF graphs and try out different alternative Endpoints such as QLever and Virtuoso and others. While typical research projects that do such subgraphing and benchmarks roll their own environments we decided to provide a python based Open Source library pyOmniGraph to provide a unified infrastructure with a standard set of REST APIs and CLI commands to improve the handling and potentially get a bigger matrix of subgraphs versus endpoints to work with. In this work we report on pyOmniGraph usage on a matrix of three RDF graphs: Wikidata, Factgrid and GOV and endpoints blazegraph, jena, QLever on a selection of subgraph combinations.

Keywords

Wikidata, FactGrid, QLever, Blazegraph, Apache Jena, GOV, RDF triple store, federated SPARQL, benchmark, PyOmniGraph

1. Introduction

Wikidata [?] has started in 2012 as an effort to back the different international Wikipedia sites with a common set of facts. Initially some 2 million entities have been harvested from existing Wikipedia content. As of 2016 over 100 million entities have been curated by a growing worldwide community of contributors. In 2022 we reported on How to get and host your own copy of wikidata.

6th Wikidata Workshop 2026, co-located with ISWC 2026, October 25–26, 2026, Bari, Italy

^{*} Preprint. Intended for submission to the 6th Wikidata Workshop 2026 (co-located with ISWC 2026); if accepted, to appear in the CEUR-WS proceedings. This version has not been peer-reviewed.

^{*}Corresponding author.

✉ wf@bitplan.com (W. Fahl)

ORCID 0000-0002-0821-6995 (W. Fahl); 0000-0003-2533-6363 (T. Holzheim); 0000-0001-9879-3827 (C. Lange); 0000-0001-6324-7164 (S. Decker)



© 2026 Copyright for this paper by its authors. Use permitted under Creative Commons License Attribution 4.0 International (CC BY 4.0).

2. Related Work

3. The PyOmniGraph Resource

3.1. Supported Backends

3.2. Command-Line Interface

4. Data Sources

5. Federated Query Benchmark

6. Evaluation

7. Availability

8. Conclusion and Future Work

Declaration on Generative AI

AI assistance in this project is limited to spelling/grammar and formatting/build tooling; AI is NOT used to generate scientific content, arguments or prose.

References

- [1] W. Fahl, T. Holzheim, C. Lange, A. Westerinen, S. Decker, Getting and hosting your own copy of Wikidata, in: *Proceedings of the 3rd Wikidata Workshop 2022*, volume 3262 of *CEUR Workshop Proceedings*, CEUR-WS.org, 2022. URL: <https://ceur-ws.org/Vol-3262/paper9.pdf>, wikidata Q115055567.
- [2] W. Fahl, pyomnigraph: Unified Python interface for multiple graph databases, <https://github.com/WolfgangFahl/pyomnigraph>, 2026. Apache-2.0, version 0.2.1.